

Commands and flows

Every command, key and menu item, and step-by-step flows of what happens inside Jarvis when you use them.

Jarvis 1.0, October 2026

In this document

- Commands you run in Terminal
- Keys, menu items and chat commands
- Quick phrases
- Flow: a spoken request, from key press to answer
- Flow: inside the brain
- Flow: a tool call and its safety checks
- Flow: tutor mode
- Flows: typed requests, the wake word, stopping and starting
- Flow: applying an update

Commands you run in Terminal

Run these from your Jarvis folder (for example `cd ~/Projects/jarvis`).

Command	What it does
<code>./setup.sh</code>	Finds Python 3.12 or 3.11, creates <code>.venv</code> , installs the packages, creates <code>.env</code> and <code>config.local.yaml</code> .
<code>./setup.sh --wakeword</code>	The same, plus the optional “Hey Jarvis” wake word.
<code>./run.sh</code>	Starts the menu-bar app (macOS).
<code>./run.sh --cli</code>	Chat with Jarvis in the terminal.
<code>./run.sh --cli --voice</code>	Talk in the terminal: press Enter on an empty line, then speak. Replies are spoken.
<code>./run.sh --listen</code>	Microphone and speech-recognition test. Needs no API key.
<code>./run.sh --say "..."</code>	Ask one question and print the answer. Add <code>--voice</code> to hear it.
<code>./run.sh --doctor</code>	Checks everything and explains how to fix problems.
<code>./run.sh --login on</code>	Start Jarvis when you log in. <code>off</code> turns it off; <code>status</code> checks it.
<code>./run.sh --version</code>	Shows the version, for example Jarvis 1.0.0 (phase 10 of 10).
<code>--model NAME</code>	Use another Claude model for this run, for example <code>--model claude-haiku-4-5-20251001</code> .
<code>--config PATH</code>	Use a different settings file instead of <code>config.yaml</code> .
<code>--debug</code>	Show detailed log messages in the terminal as well as in the log file.
<code>bash patches/<kit>/apply.sh</code>	Apply an update kit. Add <code>--commit</code> to commit or <code>--push</code> to commit and push.
<code>.venv/bin/python -m unittest discover -s tests</code>	Run the 172 tests (add <code>-v</code> to see each one).

Keys, menu items and chat commands

Anywhere on your Mac (menu-bar app)

Key	What it does
<code>^⌘C</code> (Control+Option+C)	Start talking. Press again to finish early. While Jarvis speaks, interrupts it and listens.
Hold right <code>⌘</code> Option	Talk while you hold it; release to send. Starts after a 0.3 s hold.
<code>^⌘T</code>	Tutor mode: Jarvis sees your screen and can point at things.
<code>^⌘J</code>	Type a request in a small box.
Esc	Stop listening or speaking. While Jarvis is working on a task, press it twice.
“Hey Jarvis”	Start talking, when the wake word is turned on.

The menu

Item	What it does
Status line (first line)	What Jarvis is doing right now. Not clickable.
Talk, Tutor mode, Type a request...	The same as the keys.
Stop	Stops whatever is running, with no second press.
Show panel	Brings back the floating panel.
Copy last reply	Copies Jarvis's last answer to the clipboard.
New conversation	Forgets the conversation so far. Memory facts stay.
Open workspace folder	Opens ~/JarvisWorkspace in Finder.
Edit settings	Opens <code>config.local.yaml</code> in TextEdit.
Open log	Opens <code>data/logs/jarvis.log</code> .
Quit Jarvis	Stops the hotkeys, the worker and the app.

Terminal chat commands

Command	What it does
<code>/new</code>	Start a new conversation.
<code>/memory</code>	List what Jarvis remembers about you, numbered.
<code>/forget <text or number></code>	Forget memories that match the text, or the numbered one.
<code>/usage</code>	Tokens used this session, including those read from the cache.
<code>/tutor <question></code>	Ask about your screen; Jarvis takes a screenshot.
<code>/help</code>	Show the commands.
<code>/quit</code> (or <code>/exit</code> , <code>/q</code> , <code>Ctrl+D</code>)	Leave the chat.
<code>Ctrl+C</code>	Stop Jarvis while it listens, thinks or speaks. At the prompt, leaves.

Quick phrases, spoken or typed

Say	Result
"never mind", "cancel", "stop", "forget it"	Nothing happens, and nothing is sent to Claude.
"new conversation", "start over", "reset"	"Fresh start." The conversation is cleared.
"repeat that", "say that again"	Jarvis repeats its last reply.
"thanks", "thank you", "cheers"	"Any time."
"Jarvis" on its own	"Yes, sir?" (using your <code>user_name</code>).

Flow: a spoken request

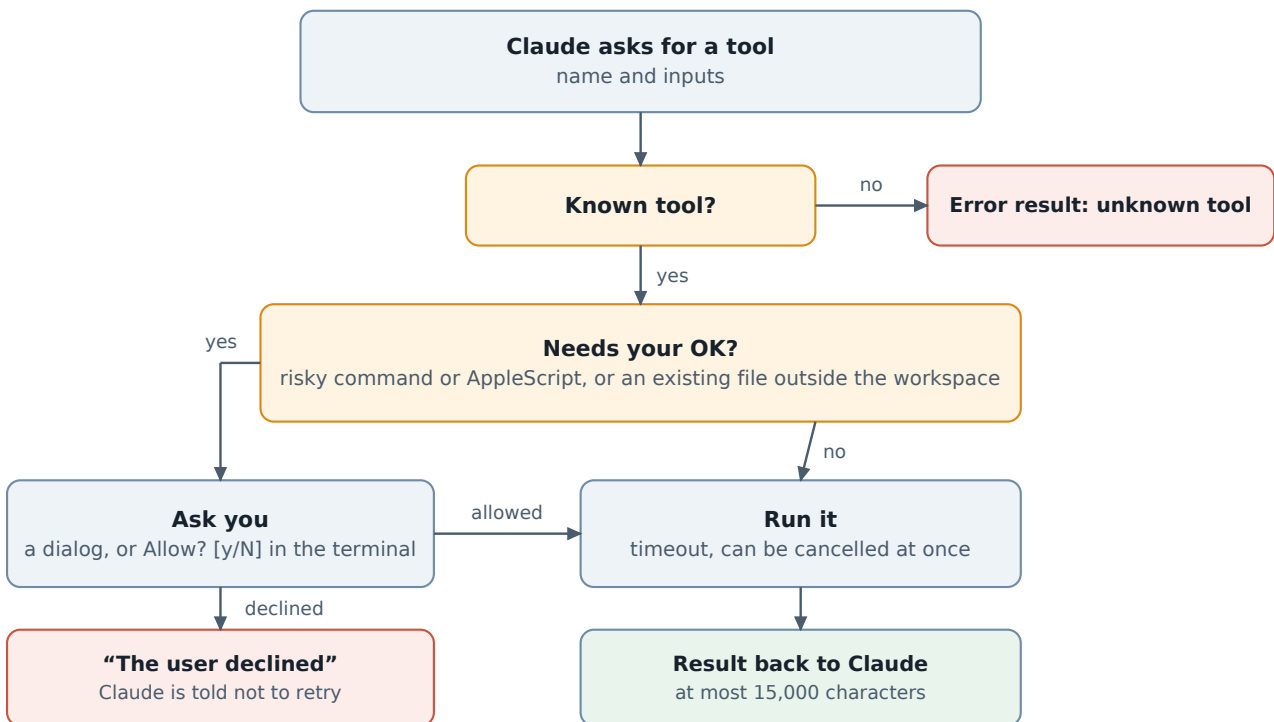


Steps 3 and 4 never leave your Mac. Step 6 sends your words, the context block and the conversation so far to Claude.

Flow: inside the brain

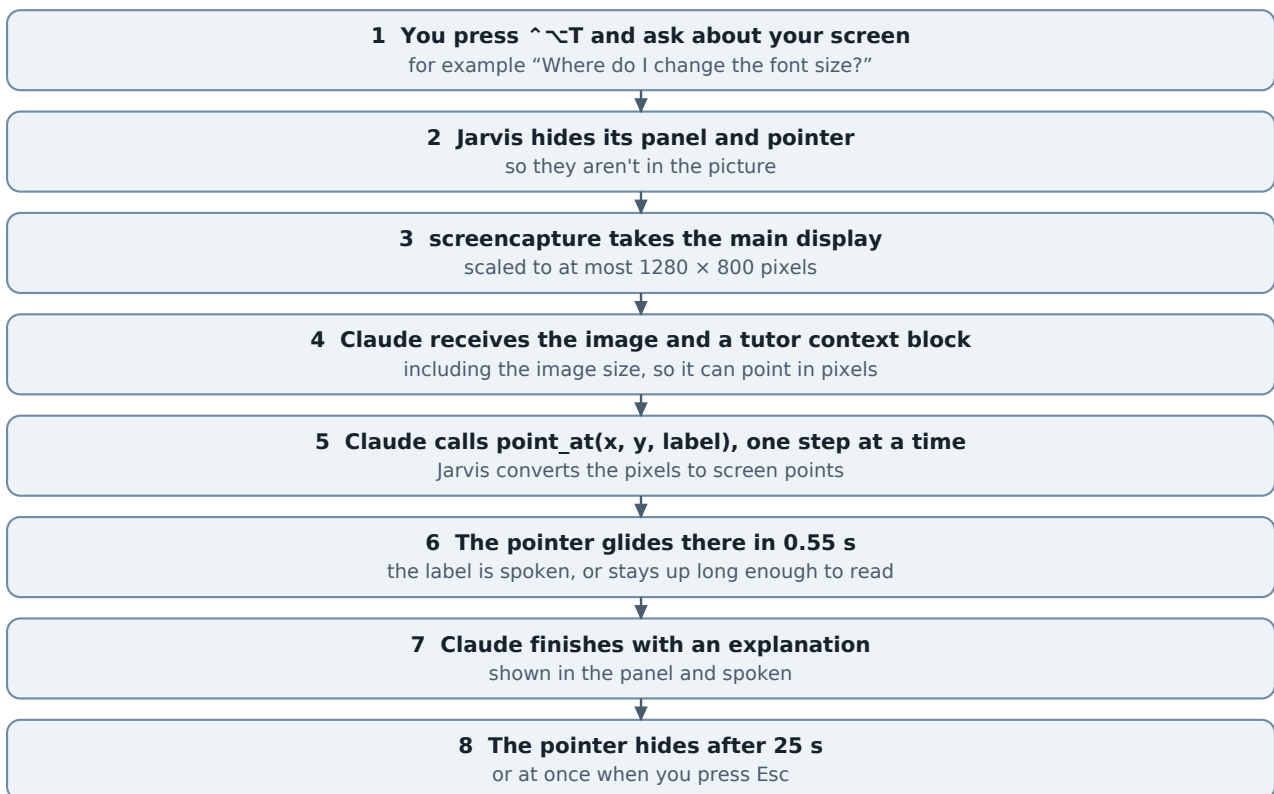
1. Your words are wrapped in the context block (time, the app you were in, how you asked, whether the reply will be spoken). Memory facts are read once for the whole turn.
2. The request is built: the model, the reply limit, the system blocks (instructions and memory, cached), the tools (installed tools plus web search) and the messages (history plus this turn).
3. The reply streams in. Text appears in the panel or terminal as it arrives; tool calls and web searches show as status lines.
4. The stop reason decides what comes next (see the System design document): answer, run tools, continue a paused search, or refuse a cut-off tool call.
5. For tool calls, each tool runs through the registry and its result is added; then the next step starts.
6. When Claude answers, the turn is saved to history (trimmed, screenshots removed) and the text is returned to be shown and spoken.
7. If something fails, you get a plain-English message; if tools had already run, the turn is saved with a note so the next request knows what happened.

Flow: a tool call and its safety checks



Protected folders are refused outright. Stopping a background job Jarvis started itself never asks. With `safety.confirm_shell` set to `always`, every command asks.

Flow: tutor mode



Other flows

A typed request (^⌘J)

1. The state becomes “waiting” and a text box opens (an AppleScript dialog).
2. What you type becomes a job with the source “typed”; Cancel or an empty box does nothing.
3. From there it follows the spoken flow from step 5, and the reply is spoken.

The wake word

1. The wake-word thread scores each 30 ms block of sound.
2. At 0.5 or more (`wake_word.threshold`) it pauses itself and calls the engine.
3. If Jarvis is idle, a listening job starts (with the chime); if Jarvis is busy, the wake word is ignored.
4. When the job ends and the queue is empty, the wake word listens again.

Stopping (Esc, Ctrl+C, Stop in the menu)

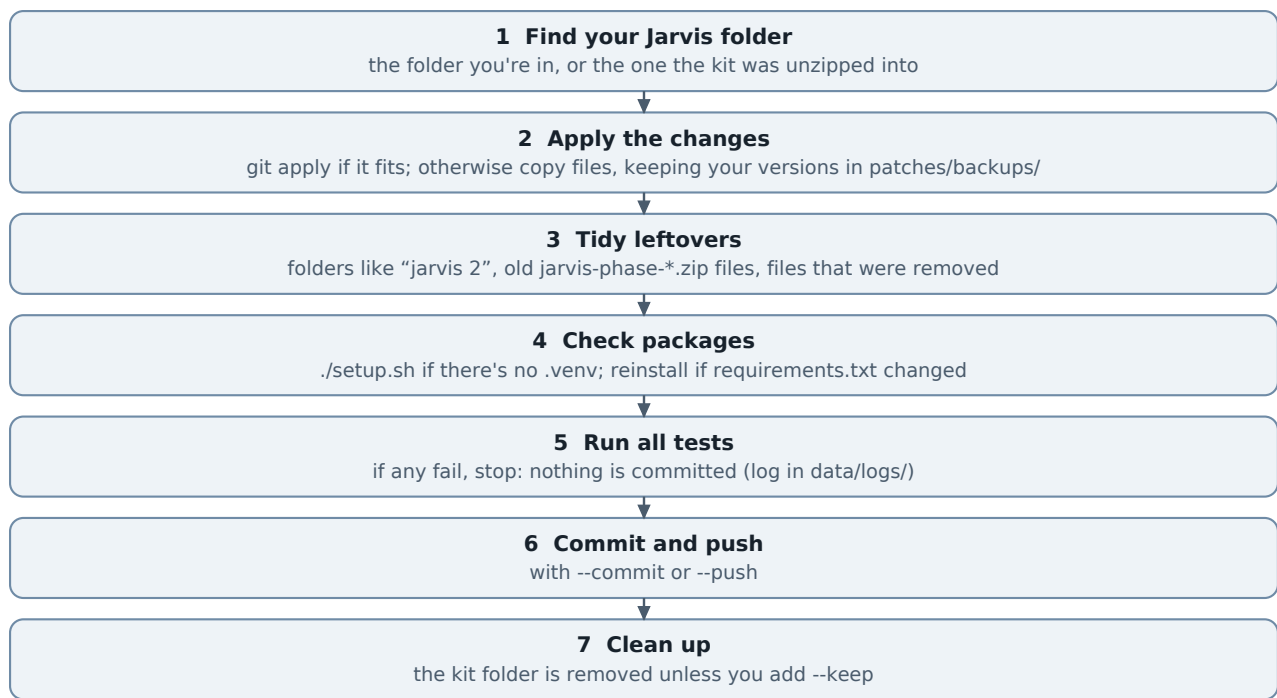
1. The cancel flag is set, any recording is aborted, speech stops and the open stream to Claude is closed.
2. Running commands are stopped with their whole process group; queued jobs are dropped.
3. The conversation is saved with a note that you stopped it, so the next request starts cleanly.

Starting the menu-bar app

1. Settings are loaded and folders created; the app hides its Dock icon.
2. The floating panel and the pointer are created (or the menu-bar fallback, if they can't start).
3. The menu and status icon appear; the worker thread starts; the microphone is opened briefly and the speech model loaded in the background.
4. The hotkeys are installed; without Accessibility, Jarvis asks once and retries every 3 s.
5. You see “Jarvis is ready” (or what's missing, such as the API key).

Flow: applying an update

```
cd ~/Projects/jarvis
unzip -oq ~/Downloads/jarvis-phase-10.zip
bash patches/phase-10/apply.sh --push
```



Running a kit twice is safe: the second run finds nothing to change.